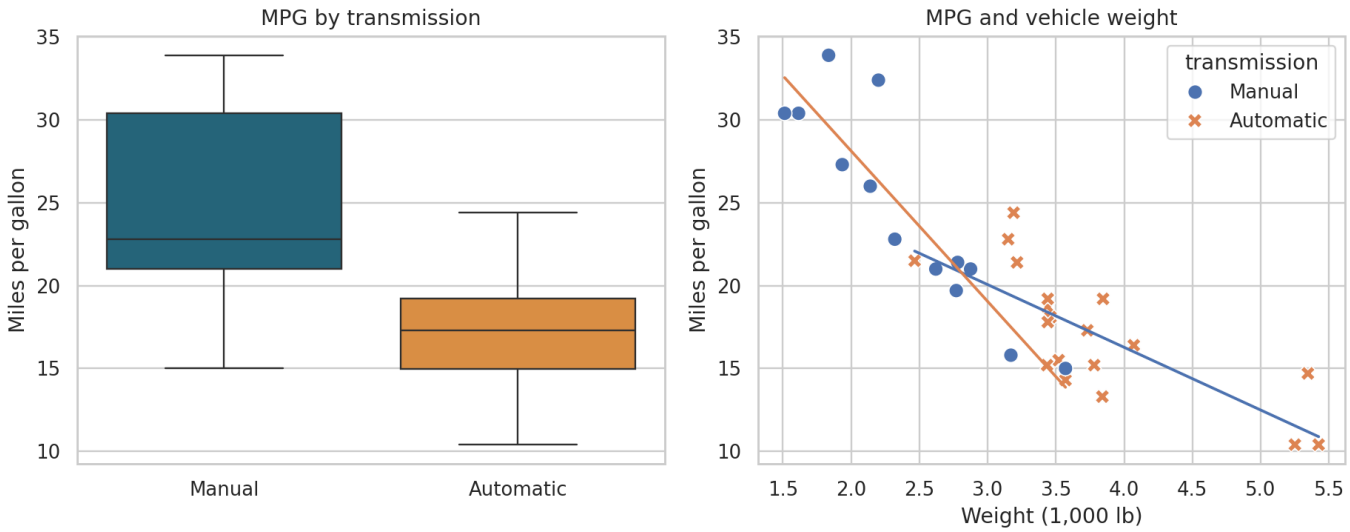
Transmission and Fuel Economy

Executive summary

The 32 cars in mtcars show a large raw MPG advantage for manual transmission. A transmission-only model estimates 7.24 MPG higher fuel economy for manual cars (95% CI 3.64 to 10.85). Because manual cars are also lighter, that comparison is confounded. Among prespecified candidates, mpg ~ am + wt + qsec had the lowest AIC. It estimates a 2.94 MPG manual advantage after adjustment (95% CI 0.05 to 5.83; p=0.047). This is an observational association and does not establish that changing transmission causes the same MPG difference.

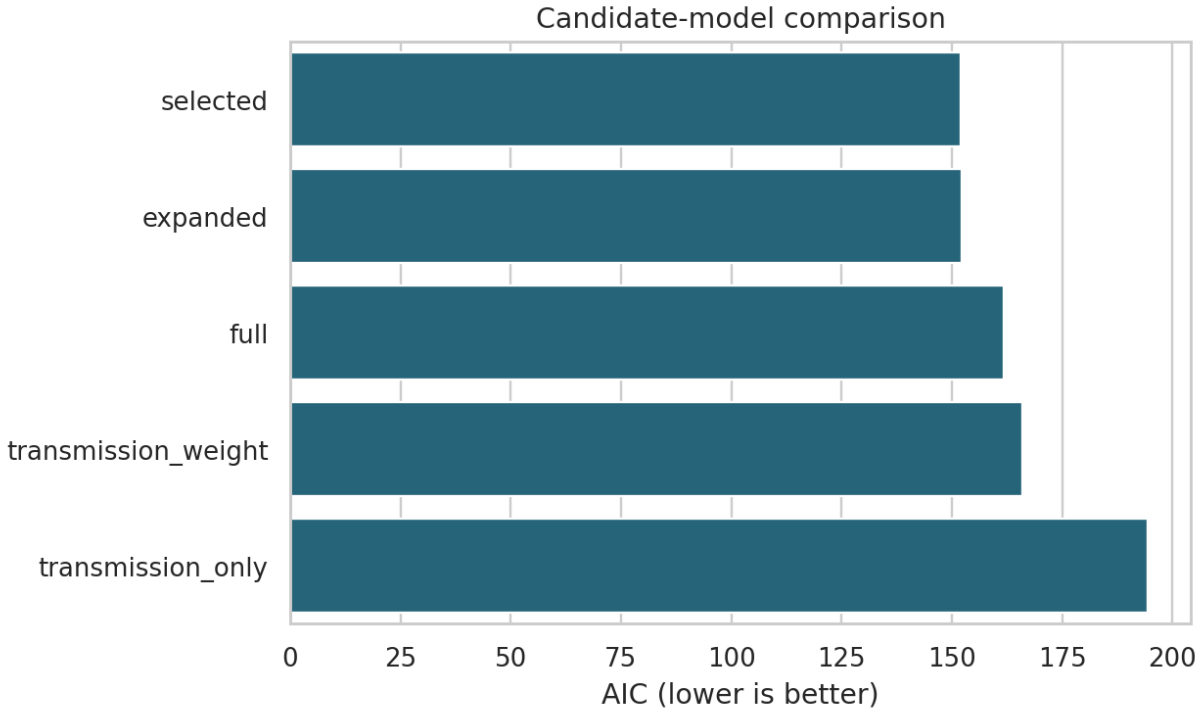
Exploratory analysis



Manual cars average 24.39 MPG versus 17.15 for automatics. The weight plot shows substantial imbalance and a strong negative MPG-weight relationship, motivating adjustment.

Model strategy and results

Transmission was retained in every candidate because it is the quantity of interest. Weight was included as the strongest engineering and graphical confounder. Quarter-mile time represents performance. Horsepower was considered in an expanded model, and the full model was checked for overfitting. The selected three-predictor model had AIC 152.12, narrowly below 152.33 for the expanded model, while using one fewer parameter.

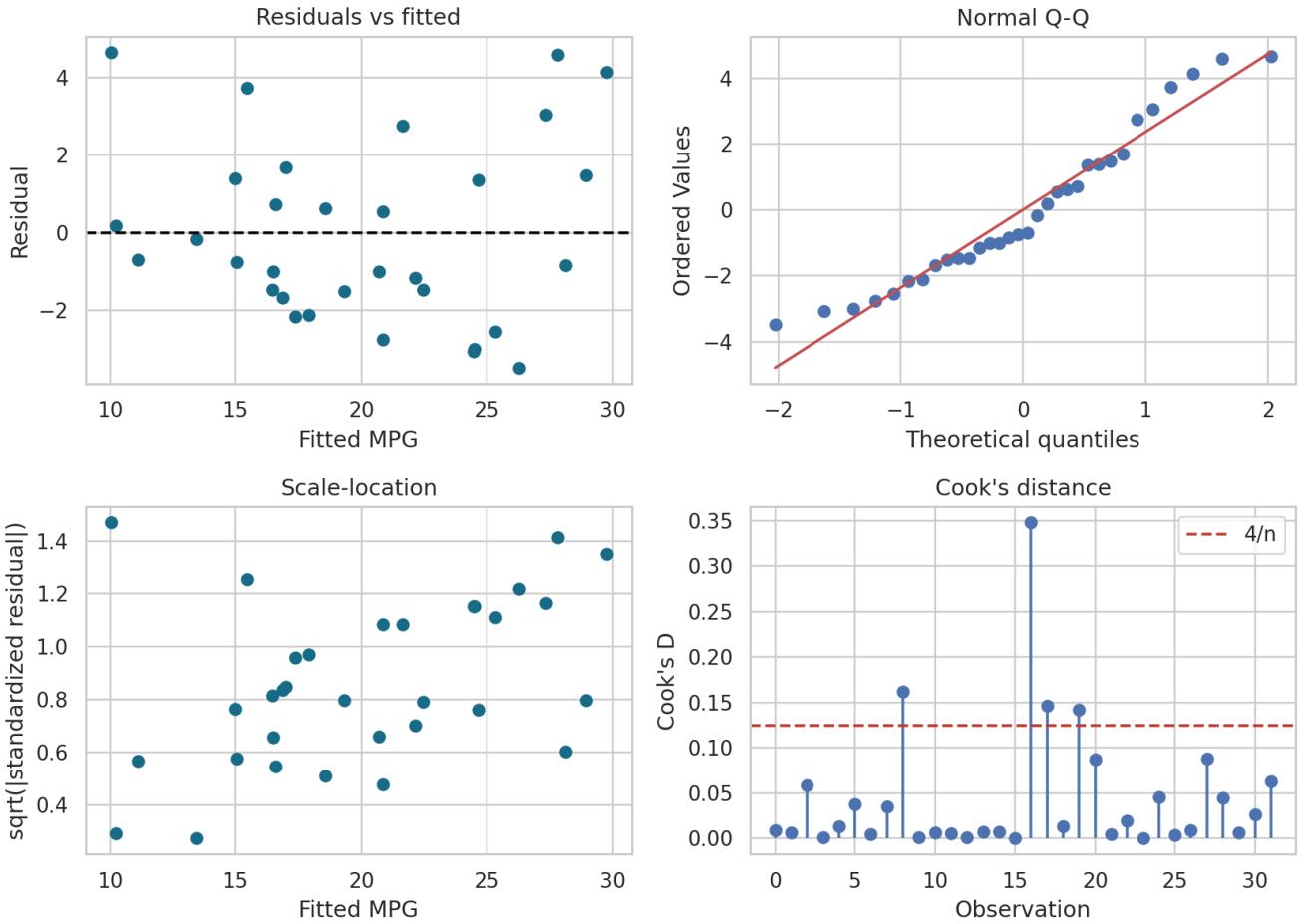


Estimate	Value	95% CI / statistic
Raw manual - automatic	7.24 MPG	[3.64, 10.85]
Adjusted manual - automatic	2.94 MPG	[0.05, 5.83]
Weight coefficient	-3.92 MPG	per 1,000 lb
Quarter-mile coefficient	1.23 MPG	per second
Adjusted R-squared	0.834	selected model

Conclusion

Manual transmission is associated with higher MPG in this sample. The adjusted interval barely excludes zero, so evidence is modest and model-dependent. The coefficient compares cars at the same weight and quarter-mile time; it should not be interpreted as a randomized causal effect.

Appendix A - Diagnostics



The Shapiro-Wilk p-value is 0.080 and the Breusch-Pagan p-value is 0.103; neither provides strong evidence against the corresponding assumptions in this small sample. Four cars exceed the 4/n Cook's-distance screen, with Chrysler Imperial largest (D=0.348). VIFs range from 1.36 to 2.54. These diagnostics support cautious use of the model while highlighting sensitivity to influential vehicles.

Appendix B - Complete analysis code

```
from pathlib import Path
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.outliers_influence import variance_inflation_factor, OLSInfluence
from scipy import stats

ROOT=Path(__file__).resolve().parent; FIG=ROOT/"figure"; FIG.mkdir(exist_ok=True)
d=pd.read_csv(ROOT/"data"/"mtcars.csv").rename(columns={"rownames":"car"})
d["transmission"]=d.am.map({0:"Automatic",1:"Manual"})

formulas={
 "transmission_only":"mpg ~ am",
 "transmission_weight":"mpg ~ am + wt",
 "selected":"mpg ~ am + wt + qsec",
 "expanded":"mpg ~ am + wt + hp + qsec",
 "full":"mpg ~ am + cyl + disp + hp + drat + wt + qsec + vs + gear + carb",
}
models={k:smf.ols(f,d).fit() for k,f in formulas.items()}
m=models["selected"]

sns.set_theme(style="whitegrid")
fig,axes=plt.subplots(1,2,figsize=(11,4.5))
sns.boxplot(data=d,x="transmission",y="mpg",hue="transmission",legend=False,palette=["#176B87","#F28E2B"],ax=axes[0])
axes[0].set(title="MPG by transmission",xlabel="",ylabel="Miles per gallon")
sns.scatterplot(data=d,x="wt",y="mpg",hue="transmission",style="transmission",s=70,ax=axes[1])
for label,sub in d.groupby("transmission"):
 p=np.polyfit(sub.wt,sub.mpg,1); x=np.linspace(sub.wt.min(),sub.wt.max(),50); axes[1].plot(x,np.polyval(p,x),lw=1.6)
axes[1].set(title="MPG and vehicle weight",xlabel="Weight (1,000 lb)",ylabel="Miles per gallon")
fig.tight_layout(); fig.savefig(FIG/"eda.png",dpi=170); plt.close(fig)

comparison=pd.DataFrame([{ "model":k,"AIC":v.aic,"BIC":v.bic,"adj_r2":v.rsquared_adj} for k,v in models.items()]).sort_values("AIC")
fig,ax=plt.subplots(figsize=(7.5,4.6)); sns.barplot(data=comparison,x="AIC",y="model",color="#176B87",ax=ax); ax.set(title="Candidate-model comparison",xlabel="AIC (lower is b

infl=OLSInfluence(m); sf=infl.summary_frame();
fig,axes=plt.subplots(2,2,figsize=(10,7.2))
axes[0,0].scatter(m.fittedvalues,m.resid,color="#176B87"); axes[0,0].axhline(0,color="black",ls="--"); axes[0,0].set(title="Residuals vs fitted",xlabel="Fitted MPG",ylabel="Re
stats.probplot(m.resid,dist="norm",plot=axes[0,1]); axes[0,1].set_title("Normal Q-Q")
axes[1,0].scatter(m.fittedvalues,np.sqrt(np.abs(sf.standard_resid)),color="#176B87"); axes[1,0].set(title="Scale-location",xlabel="Fitted MPG",ylabel="sqrt(|standardized resid
axes[1,1].stem(np.arange(len(d)),sf.cooks_d,basefmt=" "); axes[1,1].axhline(4/len(d),color="#C0392B",ls="--",label="4/n"); axes[1,1].set(title="Cook's distance",xlabel="Observ
fig.tight_layout(); fig.savefig(FIG/"diagnostics.png",dpi=170); plt.close(fig)

ci=m.conf_int(); un=models["transmission_only"]; unci=un.conf_int()
bp=het_breuschpagan(m.resid,m.model.exog)
vif={name:float(variance_inflation_factor(m.model.exog,i)) for i,name in enumerate(m.model.exog_names) if name!="Intercept"}
results={
 "rows":len(d),
 "group_summary":d.groupby("transmission").mpg.agg(["count","mean","std"]).reset_index().to_dict("records"),
```

Appendix B - Code continued

```
 "unadjusted":{"manual_minus_automatic":float(un.params.am),"ci95":unci.loc["am"].tolist(),"p":float(un.pvalues.am),"r2":float(un.rsquared)},
 "selected_formula":formulas["selected"],
 "adjusted":{"manual_minus_automatic":float(m.params.am),"ci95":ci.loc["am"].tolist(),"p":float(m.pvalues.am),"wt_coef":float(m.params.wt),"qsec_coef":float(m.params.qsec),"ad
 "candidate_models":comparison.to_dict("records"),
 "diagnostics":{"shapiro_p":float(stats.shapiro(m.resid).pvalue),"breusch_pagan_p":float(bp[1]),"max_cooks_d":float(sf.cooks_d.max()),"max_cooks_car":str(d.loc[sf.cooks_d.idxm
 "interpretation":"The adjusted transmission coefficient is an association conditional on weight and quarter-mile time; the observational data do not establish causality."
}
(ROOT/"results.json").write_text(json.dumps(results,indent=2),encoding="utf-8")
print(json.dumps(results,indent=2))
```